

Exercise 1 — Sending a Simple GET Request

Requirements

- Target any public HTTPS endpoint that returns JSON — e.g. `https://api.github.com/`.
- Build the client with `HttpClient.newHttpClient()`.
- Build the request with `HttpRequest.newBuilder()`, setting the URI to the target endpoint. GET is the default method, so you don't have to set it explicitly, though calling `.GET()` is fine too.
- Send with `client.send(request, bodyHandler)` — you must supply a body handler telling the client how to read the response body (e.g. `HttpResponse.BodyHandlers.ofString()`); `send()` won't compile without one, which is easy to miss on a first read of the API.
- Print the status code, every response header (name and value), and the body.

Expected output

- Status code: `200`.
- A handful of header lines, e.g. `content-type: application/json; charset=utf-8` and a `date` header.
- Body: the JSON GitHub's API root returns, starting with something like `{"current_user_url":"https://api.github.com/user", ...}`.

Exercise 2 — GET Request with Query Parameters

Requirements

- Prompt the user for a name from the keyboard.
- Build the request URL as `https://api.agify.io?name=<the entered name>`, URL-encoding the name (e.g. with `URLEncoder.encode(name, StandardCharsets.UTF_8)`) in case it contains spaces or other characters that aren't safe in a URL.
- Send a GET request the same way as before, reading the body as a String.
- Parse the JSON body (Jackson's ObjectMapper, or any JSON library you prefer) and pull out the `name` and `age` fields.
- Print exactly: `Name: <name>, Age: <age>`.

Expected output

- For the input `michael`: something like `Name: michael, Age: 63` — agify.io recomputes its estimate from a live dataset, so the exact number may drift over time; the point is that both fields print correctly, not that the age matches this exact value.

Exercise 3 — POST Request with a JSON Payload

Requirements

- Target `https://httpbin.org/post`.
- Build a POST request with header `Content-Type: application/json`.
- Set the body to the JSON string (e.g. `{"username":"john","password":"1234"}`) using `HttpRequest.BodyPublishers.ofString(json)`.
- Send it, read the response body as a String, and print it (parsing and pretty-printing it with your JSON library is a nice touch, but printing the raw body is enough).

Expected output

- `httpbin` echoes back a JSON object describing what it received. Abbreviated example:

```
{
  "json": { "username": "john", "password": "1234" },
  "headers": { "Content-Type": "application/json", "Host": "httpbin.org", ... },
  "url": "https://httpbin.org/post",
  "data": "{ \"username\": \"john\", \"password\": \"1234\" }",
  "origin": "<your IP>"
}
```

Exercise 4 — PUT and DELETE Requests

Requirements

- Target <https://jsonplaceholder.typicode.com/posts/1> for both requests.
- PUT: build a request with method PUT, header `Content-Type: application/json`, and a JSON body containing updated fields (e.g. a new title and body). Send it and print the status code and response body.
- DELETE: build a request with method DELETE (no body needed) to the same URL. Send it and print the status code and response body.
- JSONPlaceholder is a mock API — it doesn't actually persist any change you send, but it plays along and returns a realistic-looking response so you can practice reading one.

Expected output

- PUT: status code `200`, body echoes back the fields you sent plus the existing id, e.g. `{"id":1,"title":"...", "body":"...", "userId":1}`.
- DELETE: status code `200`, with an empty JSON object `{}` as the body. JSONPlaceholder always returns 200 for DELETE — even for a post id that doesn't exist — it never returns 204 or 404 here.

Exercise 5 — Error and Exception Handling

Requirements

- Send a GET request to a URL that returns a 4xx status, e.g. <https://api.github.com/thispagedoesnotexist>.
- An HTTP error status like 404 is **not** an exception in Java's `HttpClient` — the request completes normally and `send()` returns an ordinary `HttpResponse`. You must check `response.statusCode()` yourself and treat anything outside 200–299 as an error; only real failures (couldn't connect, DNS lookup failed, request timed out) throw an actual exception.
- To simulate a timeout reliably, regardless of your network speed, set an unrealistically short timeout on the request, e.g. `.timeout(Duration.ofMillis(1))`, against any endpoint — that should fail before a real response can come back.
- Catch `java.net.http.HttpTimeoutException` for the timeout case, and also catch the broader `java.io.IOException`, since other connection failures (e.g. an unreachable host) surface through that instead.
- Print a clear, specific message for each case — one for a non-2xx status code, a different one for a timeout/connection exception.

Expected output

- Requesting the nonexistent GitHub page: status code `404`, with a message you print yourself, e.g. `Request failed with status 404`.
 - Requesting with a 1-millisecond timeout: `send()` throws `HttpTimeoutException` before completing; your catch block should print something like `Request timed out` rather than letting a stack trace escape.
-